Me:

Hey Claude, can you please help me write the code for the portions where it detects if serves are rushed or not, along with keeping history across pages with the view history and clear history pages included?

Claude:

I'll write the rushed-serve detection plus the history, View History, and Clear History pages as one Drafter file.

I wrote `serve_history.py`, which has the rushed-serve detection plus the history, View History, and Clear History pages. I haven't run it, so test it with Drafter before you rely on it.

How it works

- **History across pages:** Drafter forgets everything between clicks except what's stored in `State`. That's why `State` holds `history`, a list of `Serve` objects (the nested data from your exam), and `last_serve_time`. Every route takes `state` in and passes it back out with `Page(state, ...)`, so the data moves from page to page with it.
- **Rushed detection:** When a serve is logged, `log_serve` calls `time.time()` to get the current time in seconds. `is_rushed` checks whether less than 5 seconds have passed since the last serve. The very first serve can't be rushed, so it uses `0.0` to mean "no serve yet."
- **Why `is_rushed` takes `now` as a parameter:** I could have called `time.time()` inside it, but then the result would change on every run and you couldn't write `assert_equal` tests for it. Passing the time in means the tests can use made-up times like 100.0 and 103.0. That should also help with your class's warning about "imposter" tests.
- **Clear History** empties the list and resets `last_serve_time` to `0.0`. Without that reset, a serve logged right after clearing could get marked as rushed.

Things you need to change

- `decide_in_or_out` is a placeholder that always returns `True`. Swap in your own in/out logic.
- The buttons point to routes by name, like `Button("Log serve", "log_serve")`. If your version of Drafter complains about that, use the function name without quotes: `Button("Log serve", log_serve)`.
- `RUSH_LIMIT` sets the 5-second cutoff, so you can change it in one place.

The main idea to take away: anything that has to last between pages goes in `State`, and keeping helpers like `is_rushed` separate from the page code is what makes them testable. If you want to read more, search "pure functions testing."

```
from drafter import *
from dataclasses import dataclass
import time

# A serve counts as "rushed" if it comes less than this many seconds after the last one
RUSH_LIMIT = 5.0


@dataclass
class Serve:
    serve_type: str
    went_in: bool
    rushed: bool


@dataclass
class State:
    history: list[Serve]
    last_serve_time: float   # 0.0 means no serve has been logged yet


# ---------- Helper functions (no Drafter pages, easy to test) ----------

def is_rushed(last_time: float, now: float) -> bool:
    """Returns True if this serve came too soon after the previous one."""
    if last_time == 0.0:          # first serve can't be rushed
        return False
    return now - last_time < RUSH_LIMIT


def add_serve(state: State, serve_type: str, went_in: bool, now: float) -> State:
    """Adds a serve to the history and remembers when it happened."""
    rushed = is_rushed(state.last_serve_time, now)
    state.history.append(Serve(serve_type, went_in, rushed))
    state.last_serve_time = now
    return state


def decide_in_or_out(serve_type: str) -> bool:
    # PLACEHOLDER: replace this with your own in/out logic
    return True


def history_rows(history: list[Serve]) -> list[list[str]]:
```

```
    """Turns the list of Serve objects into rows for a Table."""
    rows = [["#", "Serve type", "In/Out", "Rushed?"]]
    number = 1
    for serve in history:
        if serve.went_in:
            result = "In"
        else:
            result = "Out"
        if serve.rushed:
            rushed_text = "Yes"
        else:
            rushed_text = "No"
        rows.append([str(number), serve.serve_type, result, rushed_text])
        number = number + 1
    return rows


# ---------- Pages (routes) ----------

@route
def index(state: State) -> Page:
    return Page(state, [
        Header("Tennis Serve Tracker"),
        "Pick a serve type:",
        SelectBox("serve_type", ["Flat", "Slice", "Kick"]),
        Button("Log serve", "log_serve"),
        Button("View history", "view_history"),
        Button("Clear history", "clear_history"),
    ])


@route
def log_serve(state: State, serve_type: str) -> Page:
    went_in = decide_in_or_out(serve_type)
    state = add_serve(state, serve_type, went_in, time.time())
    latest = state.history[len(state.history) - 1]

    message = "Logged a " + serve_type + " serve."
    if latest.rushed:
        message = message + " That serve was RUSHED, so take your time!"

    return Page(state, [
        message,
        Button("Log another", "index"),
```

```
        Button("View history", "view_history"),
    ])


@route
def view_history(state: State) -> Page:
    if len(state.history) == 0:
        return Page(state, [
            Header("Serve History"),
            "No serves logged yet.",
            Button("Back", "index"),
        ])
    return Page(state, [
        Header("Serve History"),
        Table(history_rows(state.history)),
        Button("Clear history", "clear_history"),
        Button("Back", "index"),
    ])


@route
def clear_history(state: State) -> Page:
    state.history = []
    state.last_serve_time = 0.0   # reset so the next serve isn't marked rushed
    return Page(state, [
        "History cleared.",
        Button("Back", "index"),
    ])


# ---------- Tests ----------

# is_rushed
assert_equal(is_rushed(0.0, 100.0), False)    # first serve
assert_equal(is_rushed(100.0, 103.0), True)   # 3 seconds later = rushed
assert_equal(is_rushed(100.0, 110.0), False)  # 10 seconds later = fine
assert_equal(is_rushed(100.0, 105.0), False)  # exactly the limit = not rushed

# add_serve
assert_equal(
    add_serve(State([], 0.0), "Flat", True, 50.0),
    State([Serve("Flat", True, False)], 50.0))
assert_equal(
    add_serve(State([Serve("Flat", True, False)], 50.0), "Kick", False, 52.0),
```

```
    State([Serve("Flat", True, False), Serve("Kick", False, True)], 52.0))

# history_rows
assert_equal(history_rows([]), [["#", "Serve type", "In/Out", "Rushed?"]])
assert_equal(
    history_rows([Serve("Slice", False, True)]),
    [["#", "Serve type", "In/Out", "Rushed?"], ["1", "Slice", "Out", "Yes"]])

# clear_history
assert_equal(
    clear_history(State([Serve("Flat", True, False)], 50.0)),
    Page(State([], 0.0), [
        "History cleared.",
        Button("Back", "index"),
    ]))

start_server(State([], 0.0))
```

Me:

Can you explain how history_rows creates a table and why the header row is first

Claude:

`history_rows` takes your list of `Serve` objects and turns it into a list of lists, where each inner list is one row of the table. Drafter's `Table` component can display that kind of structure directly. It works in four steps:

1. **Start with the header.** `rows = [["#", "Serve type", "In/Out", "Rushed?"]]` creates a list that already holds one row, the column titles.
2. **Loop through the history.** `for serve in history:` visits each serve in the order it was logged.
3. **Convert each serve's data into text.** A `Serve` stores `went_in` and `rushed` as booleans (`True`/`False`). The if/else blocks turn those into readable words: `True` becomes "In" or "Yes", and `False` becomes "Out" or "No". The serve number is converted with `str(number)` so every cell in the row is a string, matching the header.
4. **Add the row and count up.** `rows.append([...])` adds the four values as a new row at the end, and `number = number + 1` makes the next serve get the next number. When the loop finishes, the function returns the whole list.

For example, if you logged a Flat serve that went in and then a rushed Kick serve that went out, the function would return:

```python
[["#", "Serve type", "In/Out", "Rushed?"],

 ["1", "Flat", "In", "No"],

 ["2", "Kick", "Out", "Yes"]]
```

Why the header row comes first

A table displays its rows in list order, top to bottom. The column titles need to appear above the data, so they have to be the first item in the list. Putting them in when the list is created also means you never need special code inside the loop to add them. Every serve after that just gets appended underneath.

It also makes each column clear. Every data row lists its values in the same order as the header (number, type, result, rushed), so each value lines up under the right title. If you ever add a column, say "Speed", you'd add it in two places: the header list and the `rows.append(...)` line, in the same position in both.

One thing to notice: since the header is always there, calling `history_rows([])` on an empty history returns just the header row. That's a good `assert_equal` test case to add:

```python
assert_equal(history_rows([]), [["#", "Serve type", "In/Out",
"Rushed?"]])
```

Me:

How can I test clear_history to make sure the most recent serve gets rested?

Claude:

You test `clear_history` like any other route. Give it a `State` that already has serves in it, call the function, and use `assert_equal` to check that it returns the page you expect. The key is to start with a `last_serve_time` that isn't `0.0`, so the test proves the reset actually happened.

Test 1: Check the whole returned page

Python

```
# clear_history
assert_equal(
    clear_history(State([Serve("Flat", True, False), Serve("Kick",
False, True)], 52.0)),
    Page(State([], 0.0), [
        "History cleared.",
        Button("Back", "index"),
    ]))
```

The input state has two serves and a most recent serve time of `52.0`. The expected page has an empty history and `last_serve_time` set back to `0.0`. If you deleted the line `state.last_serve_time = 0.0` from `clear_history`, this test would fail, because the returned state would still hold `52.0`. A test that catches that mistake is a real test, not an imposter.

Test 2: Check that the reset actually matters

The reason for resetting is so a serve logged right after clearing doesn't get marked as rushed. You can test that directly by passing the cleared state into `add_serve`:

Python

```
cleared = clear_history(State([Serve("Flat", True, False)],
50.0)).state
assert_equal(
    add_serve(cleared, "Slice", True, 52.0),
    State([Serve("Slice", True, False)], 52.0))
```

`.state` pulls the `State` back out of the `Page` that `clear_history` returns. The new serve comes only 2 seconds after the old one, which would normally count as rushed. Because the clear reset `last_serve_time` to `0.0`, `is_rushed` treats it as a first serve, and `rushed` comes out `False`.

Why both tests help

- Test 1 checks what `clear_history` does: the list is emptied and the time is reset.
- Test 2 checks why that matters: the next serve isn't wrongly marked as rushed.

One edge case worth adding: clearing a history that's already empty should still work and return the same page:

Python

```
assert_equal(
    clear_history(State([], 0.0)),
    Page(State([], 0.0), ["History cleared.", Button("Back",
"index")]))
```